

HelpMate 开发文档

仓库地址: <https://github.com/s-123489/HelpMate>

文档版本: v1.0

最后更新: 2026-06-09

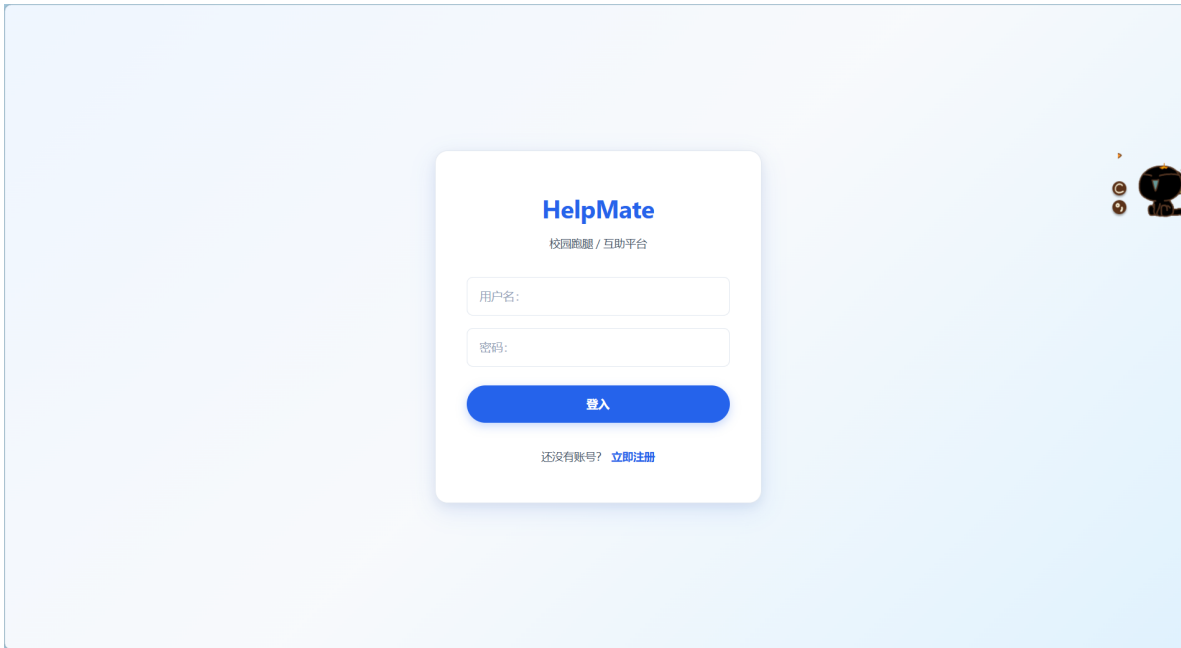
说明: 各小节标注了负责成员。

一、项目概述

本章分工: 全体成员共同撰写。

HelpMate 是一个校园互助任务平台。用户可以发布需要帮助的任务（如跑腿、借物、技能互换），其他用户可以接取任务并完成，双方通过平台内置的聊天功能沟通，任务完成后可以互相评价。平台还提供钱包功能，支持充值与任务结算。

系统采用前后端分离架构，前端基于 React 构建单页应用，后端基于 Spring Boot 提供 RESTful API，并集成 DeepSeek AI 实现智能问答功能。整个系统通过 Docker 容器化部署，使用 GitHub Actions 实现持续集成。



1.1 成员分工总览

成员	主要负责
商雨婷	UI设计（登录页设计、配色方案）、后端核心接口（用户/任务）、DeepSeek AI、后端测试、Docker后端、日志监控、安全修复
陈晓彤	UI设计（任务详情/个人中心/订单页）、前端全部基础页面、前端测试（45例，100%通过）、CI前端job、前端Docker+Nginx
李丽丽	UI设计（任务发布页）、数据库设计、API设计、聊天功能（前后端全部）、评价/个人中心重构、CI整合、Gitleaks安全扫描

二、UI 设计

本章分工： 配色方案与设计规范、登录/注册页 → 商雨婷；任务详情页/个人中心页/订单页 → 陈晓彤；任务发布页 → 李丽丽。

2.1 配色方案与设计规范（商雨婷）

全局采用蓝白主色调。主操作色为蓝色（`#1890FF`），用于主要按钮、选中态、徽标等需要引导用户注意的交互元素；背景色为白色（`#FFFFFF`）或浅灰（`#F5F5F5`），保持页面整洁；分割线与边框使用更浅的灰色（`#E8E8E8`），保持视觉层次不干扰内容。

字体规范：正文 14px，标题 18-20px，辅助说明文字 12px，行高 1.6。按钮统一圆角 8px，卡片圆角 12px，整体保持圆润感。表单控件高度统一 44px，便于触控操作。

公共组件约定：任务卡片（含类别标签、标题、悬赏金额、地点信息、发布者头像）、分类筛选按钮、金额徽标、评分星级展示，在各页面复用，保持视觉统一。

2.2 登录 / 注册页设计（商雨婷）

登录页采用居中单卡片布局，顶部展示品牌名称"HelpMate"和副标题"校园跑腿 / 互助平台"。表单包含用户名和密码两个输入框，各高 44px、圆角 8px，配有占位文字引导。主操作按钮"登入"使用全局蓝色通栏按钮，视觉权重明确。错误提示（如"用户名或密码错误"）以红色文字显示在表单顶部，不遮挡输入框。

注册页在登录页基础上增加邮箱和手机号字段，表单项从上到下排列，保持与登录页一致的视觉风格。两个页面底部均设有跳转链接，方便用户切换。

前端逻辑上，登录成功后将后端返回的 JWT Token 存入 `localStorage`（键名 `helpmate_token`），并将用户信息（id、username）存入 `localStorage`（键名 `helpmate_user`），随后导航至首页。

HelpMate

校园跑腿 / 互助平台

用户名：

密码：

登入

还没有账号？ [立即注册](#)

2.3 任务详情页 / 个人中心页 / 订单页设计 (陈晓彤)

任务详情页： 页面顶部展示任务标题、类别标签和悬赏金额，随后是任务描述、地点信息（取件地点与送达地点）和截止时间。发布者信息区域展示头像（取用户名首字）、昵称和综合评分（调用后端评价接口动态渲染）。页面下方根据当前用户身份与任务状态动态展示操作按钮：待接单时非发布者可见"接取任务"；任务进行中发布者可见"确认完成"；双方均可见"联系对方"（跳转聊天）；任务完成后双方可见"评价对方"按钮，提交后按钮变灰不可重复操作。

个人中心页： 顶部展示用户头像（首字母）、昵称、综合评分和钱包余额，底部设充值入口。中部以Tab 切换"我发布的"和"我接取的"两个任务列表，各列表支持按任务状态筛选；最下方展示当前用户收到的全部评价（星级+文字）。

订单消息页： 汇总展示用户作为接单方的所有订单（"我接的单"）和作为发布方已产生订单的任务（"我发布的"），每行展示任务标题、赏金、对方昵称和当前状态，点击可查看详情。

← 任务详情

待接取

发布时间：2026.3.14

截止时间：2026.3.20

 任务名：

 任务描述：

 取件地点：

 送达地点：

 发布者信息：



张三

★4.8

私信

¥5
赏金

接取任务

帮我取快递

进行中

发布者	李明
接单人	王五
赏金	¥5
创建时间	2026.3.14

代买午餐

待接单

发布者	李明
接单人	
赏金	¥5
创建时间	2026.3.14

打印文件

已完成

发布者	李明
接单人	张三
赏金	¥5
创建时间	2026.3.14



李明

136****1234

★4.9

⚙️ 设置

我发布的

我接取的

帮我拿快递

进行中

赏金：¥5

接单人：王五

查看详情

联系接单人

代买午餐

待接单

赏金：¥8

查看详情

打印文件

已完成

赏金：¥3

查看详情

退出登录

2.4 任务发布页设计思路（李丽丽）

任务发布页是用户将需求转化为平台任务的核心入口，设计目标是让用户以尽量少的操作完成信息填写，并在填写过程中持续给予正向引导。

类别选择可视化： 页面顶部将任务类别（跑腿、代购、代拿、代办）做成“图标+文字”的标签卡片横向排列，用户单击即可选中，比下拉菜单的视觉反馈更直接，也让用户一进入页面就明确当前要发布的任务类型。

表单分区清晰： 表单自上而下按“任务详情 → 期望完成时间 → 悬赏金额 → 补充图片”分区，每个区块配有醒目的分组标题，引导用户逐步填写、降低认知负担。任务标题标注“必填”，描述框右下角设有“0/200”字数计数器，提示用户填写适当长度的内容——越详细越容易被接单。

取送地点引导： 针对跑腿、代拿类任务“从 A 取、送到 B”的典型场景，地点拆分为“取件地点”和“送达地点”两栏，中间以“送往”连接，二者均为点击弹层选择而非手动输入，减少地址填写出错。

悬赏金额快捷填写： 金额输入框下方提供 +¥1 / +¥3 / +¥5 / +¥10 四个快捷加价按钮，用户可在自定义输入与一键加价之间自由选择，降低金额填写成本。

草稿与轻量补充： 顶栏右上角提供“草稿”入口，信息未填完时可随时保存，避免操作中断丢失；底部“补充图片（选填）”支持上传至多两张图片辅助说明，但不作为必填项，保持主流程简洁。

主操作突出： 页面底部以蓝色（全局主操作色）通栏“发布任务”按钮收尾，与全局蓝白配色保持一致，操作焦点明确。

← 发布任务

草稿📄

🏃跑腿

🛒代购

📦代拿

📅代办

任务详情

任务标题（必填）

描述任务详情，越详细越容易被接单

0/200

📍取件地点（点击选择）

>

送往

📍送达地点（点击选择）

>

期望完成时间

🕒选择截止时间

>

悬赏金额

¥输入金额

+¥1

+¥3

+¥5

+¥10

补充图片（选填）



发布任务

三、系统架构

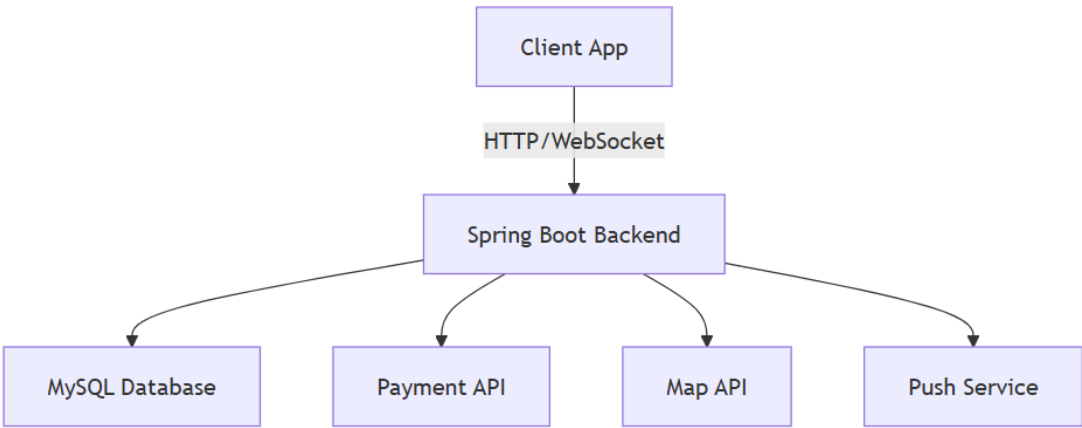
本章分工：整体架构与技术栈 → 商雨婷、陈晓彤共同；后端架构 → 商雨婷；前端架构 → 陈晓彤。

3.1 整体架构与技术栈（商雨婷、陈晓彤）

系统采用前后端分离架构。前端构建为静态单页应用（SPA），由 Nginx 托管并将 `/api` 路径的请求反向代理到后端服务；后端提供纯 RESTful API，不负责页面渲染；数据库独立部署，仅由后端访问。三个服务通过 Docker Compose 编排运行。

层次	技术选型	选型说明
前端框架	React 18 + Vite	组件化开发，Vite 提供极速本地开发与构建
路由	React Router 6	SPA 客户端路由， <code><RequireAuth></code> 守卫未登录访问
HTTP 客户端	Fetch API（封装）	统一拦截 JWT 注入、业务错误处理、日志与指标采集
后端框架	Spring Boot 3.x / Java 17	成熟生态，注解驱动，与 MyBatis-Plus 深度集成
ORM	MyBatis-Plus 3.x	零 XML 的 Lambda 条件构造器，分页插件开箱即用
数据库	MySQL 8.0	关系型存储，唯一约束用于并发接单控制
数据库迁移	Flyway	SQL 版本化迁移，保证所有环境库结构一致
认证	JWT（jjwt 库）+ BCrypt	无状态认证，密码哈希存储
AI 集成	HTTP 调用 DeepSeek 兼容接口	通过 Java HttpClient 调用，API Key 由环境变量注入
构建工具	Maven（后端）/ npm（前端）	标准工具链，CI 中使用依赖缓存加速
容器化	Docker + Docker Compose	前端、后端、MySQL 三服务统一编排
CI/CD	GitHub Actions	Push/PR 触发，前后端 job 并行执行

部署拓扑：外部请求 → Nginx（前端容器，端口 5173）→ 静态资源或 `/api` 反代 → Spring Boot（后端容器，端口 8081）→ MySQL（db 容器，端口 3307）。



3.2 前端架构（陈晓彤）

前端代码位于 `frontend/src/`，目录按职责划分：

目录/文件	职责
<code>pages/</code>	路由级页面组件，按模块子目录组织
<code>components/</code>	跨页面复用的 UI 组件（如 <code>StarRating</code> 、 <code>ReviewModal</code> 、 <code>ErrorBoundary</code> ）
<code>services/api.js</code>	封装全部后端请求，统一注入 JWT、处理业务错误码
<code>services/mockApi.js</code>	开发/测试用 Mock 层，与 <code>api.js</code> 接口一致
<code>utils/auth.js</code>	Token 与用户信息的 <code>localStorage</code> 读写工具函数
<code>utils/logger.js</code>	结构化日志工具，支持日志级别与控制台方法区分
<code>utils/metrics.js</code>	API 调用耗时、错误率等前端性能指标收集
<code>utils/healthCheck.js</code>	前端自身与后端可用性探测
<code>App.jsx</code>	路由配置入口， <code><RequireAuth></code> 包裹需鉴权的路由

路由共10条，登录（`/login`）和注册（`/register`）为公开路由，其余均由 `<RequireAuth>` 守卫——若 `localStorage` 中无有效 Token，自动重定向到登录页。路由与页面的对应关系如下：`/`（首页）、`/task/:id`（任务详情）、`/task/publish`（发布任务）、`/order/message`（订单消息）、`/chat`（会话列表）、`/chat/:userId`（聊天房间）、`/user/center`（个人中心）、`/ai/chat`（AI 客服）。

`api.js` 中的 `request` 函数统一处理：从 `localStorage` 取 Token 并注入 `Authorization: Bearer <token>` 请求头；响应后检查后端统一返回格式（`{ code, message, data }`），若 `code !== 200` 则抛出 `Error`；同时记录每次请求的耗时和状态码到 `metrics` 模块，并写入结构化日志。

```

frontend/
├── pages/                # 页面
│   ├── Auth/            # 认证相关页面
│   ├── Task/            # 任务相关页面
│   ├── User/            # 用户相关页面
│   └── Wallet/          # 钱包相关页面
├── components/          # 组件
│   ├── TaskCard/        # 任务卡片
│   ├── UserInfo/        # 用户信息
│   └── Map/             # 地图组件
├── utils/               # 工具函数
│   ├── api.js           # API 调用
│   ├── storage.js       # 本地存储
│   └── location.js      # 位置服务
├── services/            # 服务
│   ├── auth.js          # 认证服务
│   ├── task.js          # 任务服务
│   ├── user.js          # 用户服务
│   └── wallet.js        # 钱包服务
├── navigation/          # 导航配置
└── App.js               # 应用入口

```

3.3 后端架构（商雨婷）

后端代码位于 `backend/src/main/java/com/helpmate/`，严格遵循分层架构：

层次	包名	职责
接口层	<code>controller</code>	接收 HTTP 请求，做参数校验，调用 Service，封装响应
业务逻辑层	<code>service</code>	核心业务规则，事务控制，调用 Mapper
数据访问层	<code>mapper</code>	MyBatis-Plus Mapper 接口，继承 <code>BaseMapper<T></code>
数据传输对象	<code>dto</code>	接收前端请求体的字段对象（不暴露数据库实体）
视图对象	<code>vo</code>	返回给前端的响应字段对象（屏蔽敏感字段）
实体类	<code>entity</code>	与数据库表——对应的 Java 对象
公共组件	<code>common</code>	JWT 工具、拦截器、全局异常处理、CORS 配置、MyBatis-Plus 配置

统一返回结构：所有接口通过 `Result<T>` 泛型类封装响应，固定字段为 `code`（状态码）、`message`（提示信息）、`data`（业务数据）。成功时 `code` 为 200，参数错误为 400，未认证为 401，服务器错误为 500。

JWT 认证流程： `JwtUtil` 使用 HMAC-SHA256 算法签发和验签 Token，密钥与过期时间通过 `application.yml` (`jwt.secret`、`jwt.expiration`) 注入，CI 中通过环境变量覆盖，不硬编码。`AuthInterceptor` 实现 `HandlerInterceptor`，在 `preHandle` 阶段从 `Authorization` 头提取 Token 并验证，验证通过后将 `userId` 存入 `request` 属性，后续 Controller 直接通过 `request.getAttribute("userId")` 获取，无需每个接口重复解析 Token。拦截器排除路径包括登录、注册、AI 问答、任务列表和健康检查端点。

全局异常处理： `GlobalExceptionHandler` 通过 `@RestControllerAdvice` 统一捕获 `MethodArgumentNotValidException` (参数校验失败)、`BindException`、`ConstraintViolationException` 和 `RuntimeException`，分别映射为 400 或 500 响应，Controller 层无需 try-catch。

Flyway 与 MyBatis-Plus： 应用启动时 Flyway 自动按版本序号执行未执行的迁移脚本，`MybatisPlusConfig` 注册分页插件 (`PaginationInnerInterceptor`)，`MyMetaObjectHandler` 自动填充 `createdAt`/`updatedAt` 字段。

```
backend/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/helpmate/
│   │   │       ├── controller/  # 控制层
│   │   │       ├── service/    # 业务层
│   │   │       ├── mapper/     # 数据访问层
│   │   │       └── entity/     # 实体类
│   │   └── resources/
│   │       └── application.yml  # 配置文件
└── pom.xml
```

四、数据库设计

本章分工： 全部由李丽丽负责。

4.1 设计原则

数据库设计遵循第三范式 (3NF)，减少数据冗余。所有表均设有 `created_at`、`updated_at` 时间戳字段，由 `MyMetaObjectHandler` 自动填充 (见第六章)。逻辑删除通过 `is_deleted` 字段实现，避免物理删除造成关联数据丢失。

4.2 Flyway 版本迁移策略

数据库变更通过 Flyway 进行版本化管理，分三个版本迁移：

- **V1：** 核心4张表——`user`、`task`、`order_info`、`wallet_transaction`
- **V2：** 新增 `review` (评价)、`location_record` (位置)、`notification` (通知)
- **V3：** 新增 `chat_session` (聊天会话)、`chat_message` (聊天消息)

每次表结构变更写成对应版本的 SQL 文件，团队成员拉代码后启动项目即自动按顺序执行，保证所有人和生产环境的库结构完全一致。

4.3 核心表结构

用户表（user）

存储系统用户的基本信息与认证凭据。`password_hash` 存储 BCrypt 加密后的哈希值，原始密码不落库。

字段名	类型	说明
<code>id</code>	BIGINT, PK	用户唯一标识，自增
<code>username</code>	VARCHAR(50)	用户名，唯一索引 <code>uk_username</code>
<code>password_hash</code>	VARCHAR(255)	BCrypt 加密后的密码
<code>nickname</code>	VARCHAR(50)	展示昵称
<code>avatar_url</code>	VARCHAR(255)	头像路径
<code>phone</code>	VARCHAR(20)	手机号，唯一索引 <code>uk_phone</code>
<code>rating_score</code>	DECIMAL(3,2)	综合评分，默认5.00
<code>balance</code>	DECIMAL(10,2)	钱包余额快照
<code>is_deleted</code>	TINYINT(1)	逻辑删除标志
<code>created_at</code>	DATETIME	注册时间（自动填充）
<code>updated_at</code>	DATETIME	最后更新时间（自动填充）

任务表（task）

字段名	类型	说明
<code>id</code>	BIGINT, PK	任务唯一标识
<code>publisher_id</code>	BIGINT	发布者用户ID，索引 <code>idx_publisher_id</code>
<code>title</code>	VARCHAR(100)	任务标题
<code>description</code>	TEXT	任务描述
<code>category</code>	VARCHAR(50)	类别（EXPRESS/FOOD/PURCHASE/OTHER）
<code>reward</code>	DECIMAL(10,2)	悬赏金额
<code>deadline</code>	VARCHAR(50)	截止时间
<code>status</code>	TINYINT	0待接单 / 1进行中 / 2已完成 / 3已取消
<code>created_at</code>	DATETIME	发布时间

订单表 (order_info)

task_id 加唯一约束 uk_task_id，保证一个任务只能被接一次。两人并发接单时，数据库层直接让第二条插入失败，从根本上防止重复接单，不依赖应用层判断。

字段名	类型	说明
id	BIGINT, PK	订单唯一标识
task_id	BIGINT	关联任务ID, 唯一约束 uk_task_id
helper_id	BIGINT	接取者用户ID, 索引 idx_helper_id
status	TINYINT	0进行中 / 1已完成 / 2已取消
created_at	DATETIME	接单时间

钱包流水表 (wallet_transaction)

用一个 amount 字段正负表示入账/出账，余额等于所有流水之和，对账天然自治；type 字段标记业务含义，比拆成收支两个字段更简洁。

字段名	类型	说明
id	BIGINT, PK	流水唯一标识
user_id	BIGINT	用户ID
amount	DECIMAL(10,2)	金额，正数入账 / 负数出账
type	TINYINT	1充值 / 2提现 / 3任务支付 / 4接单收入
remark	VARCHAR(200)	流水备注
created_at	DATETIME	发生时间

4.4 聊天相关表 (V3新增，李丽丽设计)

聊天会话表 (chat_session)

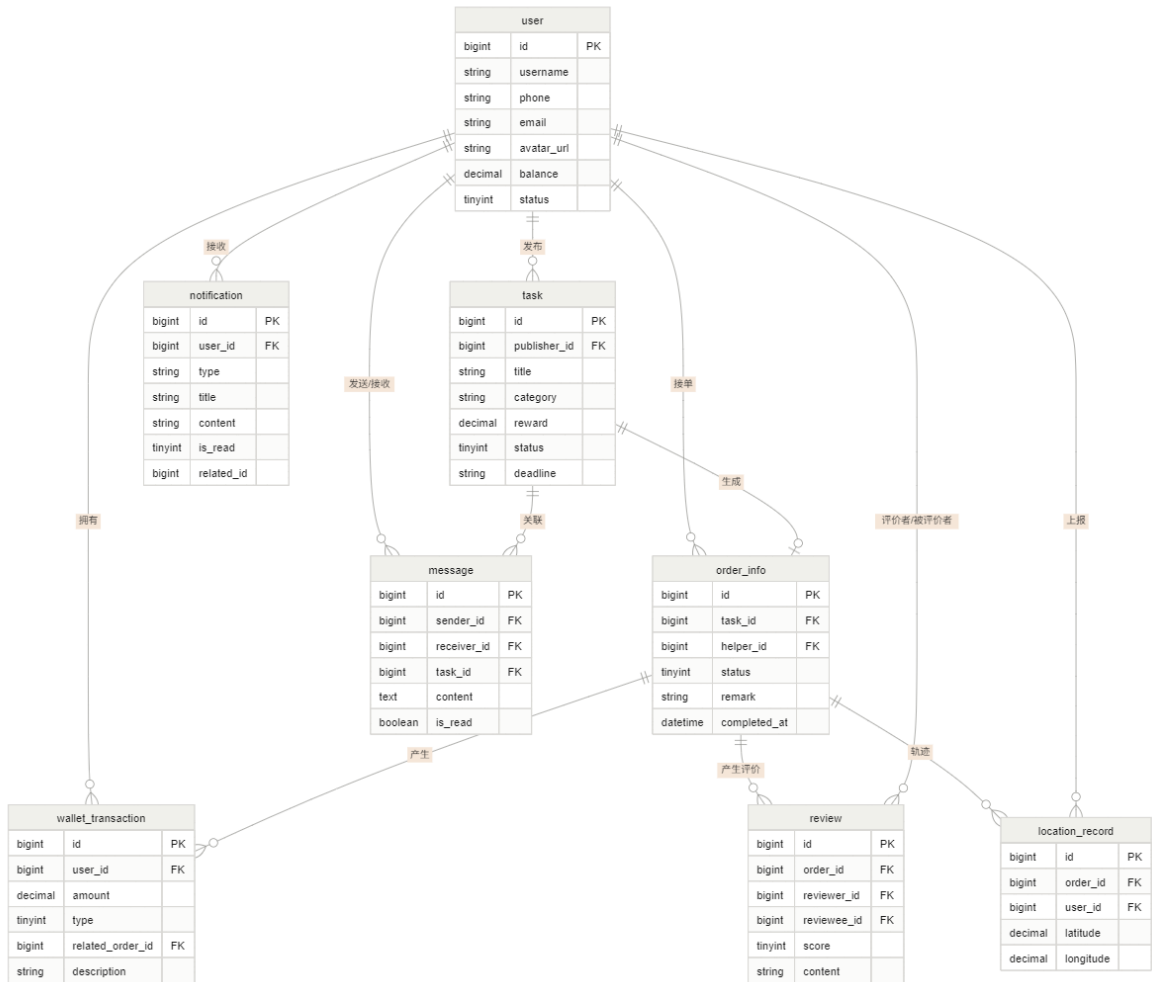
未读数拆分为 unread_count_publisher 和 unread_count_acceptor 两个字段，而非存在消息表。查询会话列表时只需读取会话表，无需聚合消息表，减少查询开销。

字段名	类型	说明
id	BIGINT, PK	会话唯一标识
task_id	BIGINT	关联任务ID
publisher_id	BIGINT	发布者用户ID
acceptor_id	BIGINT	接收者用户ID
last_message	VARCHAR(255)	最新消息摘要
last_message_at	DATETIME	最新消息时间
unread_count_publisher	INT	发布者未读数
unread_count_acceptor	INT	接收者未读数
created_at	DATETIME	会话创建时间

聊天消息表（chat_message）

session_id 建有索引，加速按会话查询消息列表。

字段名	类型	说明
id	BIGINT, PK	消息唯一标识
session_id	BIGINT	所属会话ID，索引
sender_id	BIGINT	发送者用户ID
content	TEXT	消息内容
created_at	DATETIME	发送时间



五、API 设计

本章分工：全部由李丽丽负责。

5.1 设计规范

API 遵循 RESTful 风格，资源以复数名词命名，HTTP 方法按语义使用：`GET` 查询、`POST` 创建、`PUT / PATCH` 更新、`DELETE` 删除。所有接口统一返回以下结构的 JSON 响应：

```
{
  "code": 200,
  "message": "操作成功",
  "data": {}
}
```

失败时 `code` 为对应错误码 (400/401/500) , `data` 为 `null` , `message` 为具体错误信息。

身份认证采用 JWT 方案：用户登录后服务端颁发 Token，客户端存入 `localStorage`，后续请求通过 `axios` 拦截器自动在 `Authorization: Bearer <token>` 头中携带；后端通过统一拦截器 `AuthInterceptor` 校验，无需每个接口单独写认证逻辑。

5.2 用户认证 API

方法	路径	功能	需认证
POST	/api/user/register	用户注册	否
POST	/api/user/login	用户登录, 返回 JWT Token	否

注册接口设计说明: 请求体包含 `username`、`password`、`email`、`phone`, 用户名为必填。Service 层先检查用户名是否已存在, 若重复返回 500 ("用户名已存在"); 否则对密码进行 BCrypt 加密后存库, 余额默认为零。

登录接口设计说明: 为防止账号枚举攻击, 无论用户名不存在还是密码错误, 均统一返回"用户名或密码错误", 不区分具体原因。

```
// POST /api/user/login 成功响应示例
{
  "code": 200,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "userId": 1,
    "username": "lili"
  }
}
```

5.3 任务管理 API

方法	路径	功能	需认证
GET	/api/task/list	获取任务列表 (支持分页、类别筛选)	是
POST	/api/task/create	发布新任务	是
GET	/api/task/{id}	获取任务详情	是
POST	/api/task/{id}/cancel	取消任务 (仅发布者, 仅待接单状态)	是
GET	/api/task/my-tasks	获取我发布的所有任务	是
POST	/api/order/accept	接单任务 (body: taskId)	是
POST	/api/order/{id}/complete	确认任务完成 (仅发布者)	是
POST	/api/order/{id}/cancel	取消订单 (发布者或接单者均可)	是
GET	/api/order/my-orders	我接的单列表	是
GET	/api/order/my-published	我发布任务对应的订单列表	是

发布任务时 Service 层在同一事务中调用 `walletService.freezeReward`, 从发布者余额中扣除悬赏金额; 任务取消或订单取消时通过 `walletService.refundReward` 退还; 任务完成时通过 `walletService.releaseReward` 将奖金转给接单者。

5.4 新增扩展接口

在开发聊天、评价、个人中心功能时（李丽丽负责），新增了以下接口：

模块	接口路径（前缀 /api）
消息	POST /message/send、GET /message/conversation/{otherId}、GET /message/conversations
评价	POST /review/submit、GET /review/user/{userId}、GET /review/profile/{userId}、GET /review/profile/me
钱包	GET /wallet/balance、POST /wallet/recharge、POST /wallet/withdraw、GET /wallet/transactions
通知	GET /notification/list、POST /notification/{id}/read、POST /notification/read-all

5.5 Apifox 接口测试

李丽丽在 Apifox 中编写了5个测试用例：用户注册成功、用户名重复注册（期望500）、登录成功、密码错误（期望500）、未登录访问受保护接口（期望401）。

POST

 /api/user/login

发送

保存

 删除

Params

Body 1

Headers

Cookies

Auth

前置操作

...

</>

none

form-data

x-www-form-urlencoded

JSON

XML

Text

Binary

 自动生成



```
1 {
2   "username": "lilili",
3   "password": "123456"
4 }
```

...

Body

Cookie

Header

...

200 · 91ms · 318B ·  

 通过

pretty

JSON

utf8



```
1 {
2   "code": 84,
3   "message": "Ut",
4   "data": {
5     "token": "in et",
6     "userId": 59,
7     "username": "盍国荣"
8   }
9 }
```

六、后端开发

本章分工： 用户模块、任务模块、DeepSeek AI 功能集成 → 商雨婷；MyMetaObjectHandler、聊天/评价/钱包接口、CORS 修复 → 李丽丽。

6.1 用户模块（商雨婷）

用户模块包含注册和登录两个核心功能，均在 `UserServiceImpl` 中实现。

注册：Service 层首先通过 `LambdaQueryWrapper` 按用户名查询数量，若已存在则直接抛出运行时异常，由全局异常处理器统一返回 500 错误。用户名不存在时，使用 Spring Security 内置的 `BCryptPasswordEncoder` 对明文密码进行哈希加密（默认 strength 10，不可逆），只将哈希值存入 `password_hash` 字段，原始密码不落库。新用户的初始余额设为零、状态设为1（正常）。

登录：同样用 `LambdaQueryWrapper` 按用户名查找用户；若用户不存在，或通过 `passwordEncoder.matches(明文, 哈希)` 验证密码不匹配，均统一返回"用户名或密码错误"，故意不区分两种情况，防止攻击者通过错误信息枚举有效账号。验证通过后调用 `JwtUtil.generateToken(userId, username)` 生成 Token，将 token、userId、username 封装为 `LoginVO` 返回前端。

`JwtUtil` 使用 `jjwt` 库，签名算法为 HMAC-SHA256，密钥和过期时间从 `application.yml` 注入（生产环境通过环境变量 `JWT_SECRET` 覆盖）。`getUserIdFromToken` 方法解析 Token 的 `subject` 字段即可获取 userId，后续所有需要身份的接口均通过此方法获取当前用户。

6.2 任务模块（商雨婷）

任务模块涵盖任务的完整生命周期：发布 → 待接单 → 进行中 → 已完成/已取消。

任务发布：`createTask` 方法标注 `@Transactional`，将 `CreateTaskRequest` 的字段映射为 `Task` 实体，设初始状态为0（待接单），插入数据库后立即调用 `walletService.freezeReward` 从发布者余额中扣除悬赏金额。若余额不足，事务回滚，任务不被创建。

任务列表：`listTasks` 用 MyBatis-Plus 分页插件（`Page<Task>`）查询，`LambdaQueryWrapper` 筛选 `status=0`（待接单），支持按 `category` 条件过滤，按创建时间倒序。查出任务列表后批量查询发布者信息，组装为 `TaskListVO` 返回，前端无需二次请求用户信息。

接单控制：`OrderServiceImpl.acceptOrder` 方法先检查任务状态是否为0，再查询 `order_info` 表中是否已有该 `taskId` 的记录，双重检查后插入订单。数据库层的唯一约束 `uk_task_id` 作为最终防线——并发场景下第二个请求插入时会触发唯一键冲突异常，由全局异常处理器捕获返回"任务已被接单"，彻底杜绝一个任务被多人接取的情况。

任务取消：只有发布者可以取消，且只有在待接单状态（`status=0`）时可以取消。取消后状态置为3，并调用 `walletService.refundReward` 退还冻结的赏金。

完成任务：只有发布者可以确认完成，状态需为进行中（`status=0` 的订单）。完成后订单状态置1、任务状态置2，调用 `walletService.releaseReward` 将赏金转给接单者，同时通过 `NotificationService` 向双方推送通知。

6.3 DeepSeek AI 功能集成（商雨婷）

AI 智能客服功能集成在 `AIController` 和 `AIServiceImpl` 中，通过 HTTP 调用兼容 OpenAI Chat Completions 格式的 DeepSeek API（实际使用 SiliconFlow 托管的 `Qwen/Qwen2.5-7B-Instruct` 模型）。

接入方式：`AIServiceImpl` 使用 Java 17 内置的 `HttpClient` 发起 POST 请求，连接超时10秒、读取超时30秒。请求体构造为标准的 `messages` 数组格式，包含一条 `system` 角色的固定提示词和用户输入的 `user` 消息。

提示词设计：系统提示词将 AI 定位为"HelpMate 校园跑腿互助平台的智能客服"，要求用简洁友好的语言回答，若话题与平台无关则礼貌引导用户回到平台相关话题。提示词按业务模块分节，涵盖平台简介、账号相关（注册字段、登录 Token 有效期24小时）、任务相关（发布流程与赏金冻结逻辑、取消限制、四种任务状态说明）、接单相关（筛选浏览、接单限制、确认完成与赏金结算、取消订单流程）、钱包相关（余额查询、充值与提现规则、五类流水说明）、评价相关（1-5星+文字、每人只能评一次、综合评分计算）、聊天相关（会话列表、未读数）、通知相关（接单/完成/取消/评价等系统通知），以及常见问题汇总（余额不足、接单后无法单方删除、只有发布者可确认完成等）。详细的分节内容使 AI 能够精准回答用户各类操作问题，减少无效引导。

API Key 管理： apiKey、baseUrl、model 三个字段通过 @value 从 application.yml 读取，yml 中的值来自环境变量（AI_API_KEY、AI_BASE_URL、AI_MODEL），CI 中注入 dummy-key-for-ci 占位，生产和开发环境通过 compose.yaml 的 environment 段注入真实值，源码中无任何硬编码凭据。

异常处理：若后端 AI 服务返回非200状态码，或网络调用抛出异常，均统一转换为"AI 服务暂时不可用，请稍后重试"的 RuntimeException，由全局异常处理器返回500，前端展示友好提示。

6.4 MyMetaObjectHandler（时间戳自动填充）（李丽丽）

实现了 MyMetaObjectHandler.java，用于 MyBatis-Plus 的字段自动填充。所有实体类中标注 @TableField(fill = FieldFill.INSERT) 的 createTime 和 @TableField(fill = FieldFill.INSERT_UPDATE) 的 updateTime 字段，在插入和更新时由该处理器自动注入当前时间，无需在每个 Service 方法中手动赋值。

6.5 聊天后端接口（李丽丽）

后端新增以下消息相关接口，均需 JWT 认证：

方法	路径	功能
GET	/api/messages/sessions	获取当前用户所有会话（含未读数）
GET	/api/messages/sessions/{sessionId}	获取某会话的消息列表
POST	/api/messages/send	发送消息
PUT	/api/messages/sessions/{sessionId}/read	将会话标记为已读

发送消息时，Service 层在写入 chat_message 表的同时，更新 chat_session 的摘要字段，并将接收方的未读计数加1。标记已读时将当前用户对应的未读字段归零。

```
// 发送消息核心逻辑（伪代码）
void sendMessage(Long sessionId, Long senderId, String content) {
    // 1. 写入消息记录
    chatMessageMapper.insert(new ChatMessage(sessionId, senderId, content));
    // 2. 更新会话摘要
    session.setLastMessage(content);
    session.setLastMessageAt(LocalDateTime.now());
    // 3. 接收方未读数 +1
    if (senderId.equals(session.getPublisherId())) {
        session.setUnreadCountAcceptor(session.getUnreadCountAcceptor() + 1);
    } else {
        session.setUnreadCountPublisher(session.getUnreadCountPublisher() + 1);
    }
}
```

```
chatSessionMapper.updateById(session);  
}
```

6.6 评价与钱包接口（李丽丽）

评价接口： `POST /api/evaluations` 提交评价时，后端校验同一任务同一用户只能评价一次，防止重复提交。评价写入后重新计算被评价方的综合评分并更新 `user.rating_score`。

钱包接口： `POST /api/wallet/recharge` 充值时，在同一事务中同时更新 `user.balance` 字段和插入 `wallet_transaction` 流水记录，保证余额与流水一致。提现时额外检查余额是否充足，不足则抛出异常。流水类型用整数编码（1充值/2提现/3赏金冻结/4接单收入/5退款），Service 内置 `TYPE_DESC` 映射表转换为可读文字后返回前端。

6.7 CORS 预检请求修复（李丽丽）

前端的 `PUT`、`DELETE` 等非简单请求在发出前会先发送 `OPTIONS` 预检请求。原有配置未放行 `OPTIONS`，导致请求被 JWT 拦截器拦截返回 401。李丽丽在 CORS 配置中添加了对 `OPTIONS` 方法的支持，并在拦截器排除路径中加入 `OPTIONS` 请求的跳过逻辑，修复了联调时的跨域问题。

七、前端开发

本章分工： 全部基础页面实现 → 陈晓彤；聊天功能、评价重构、个人中心重构 → 李丽丽。

7.1 前端基础页面实现（陈晓彤）

前端共实现以下基础页面：

登录页（Login.jsx）： 居中单卡片布局，用户名+密码表单，点击"登入"后调用 `api.login`，成功后将 Token 和用户信息写入 `localStorage` 并跳转首页，失败时在表单顶部展示错误提示。

注册页（Register.jsx）： 字段比登录页多出邮箱和手机号，调用 `api.register`，注册成功后跳转登录页。

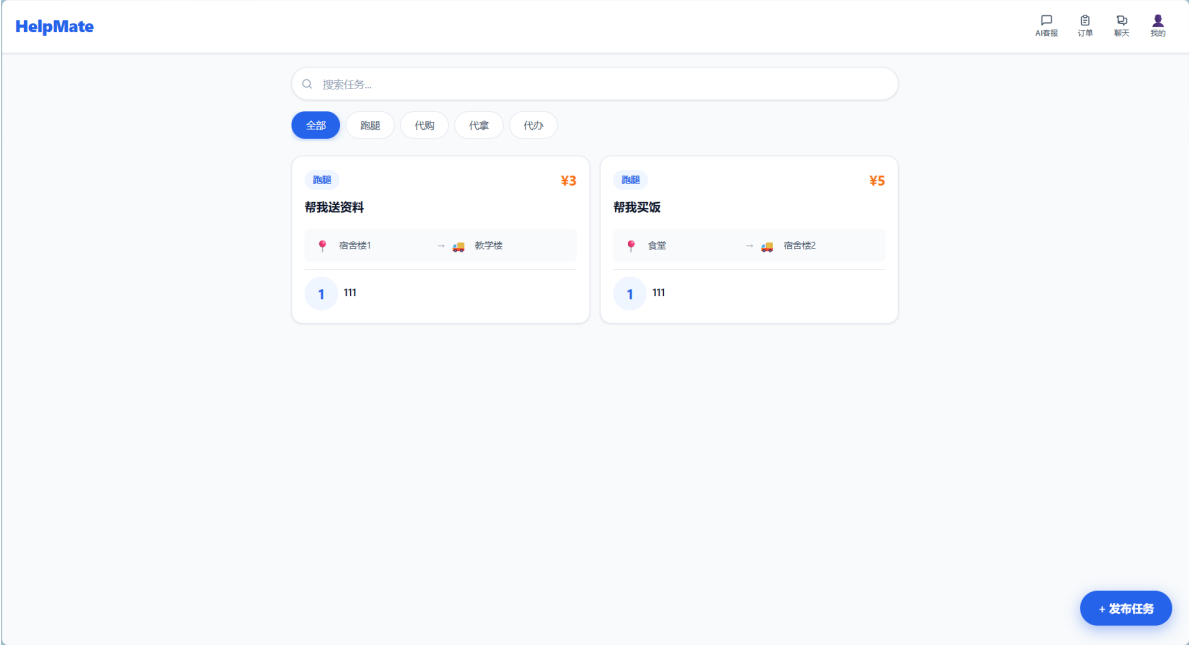
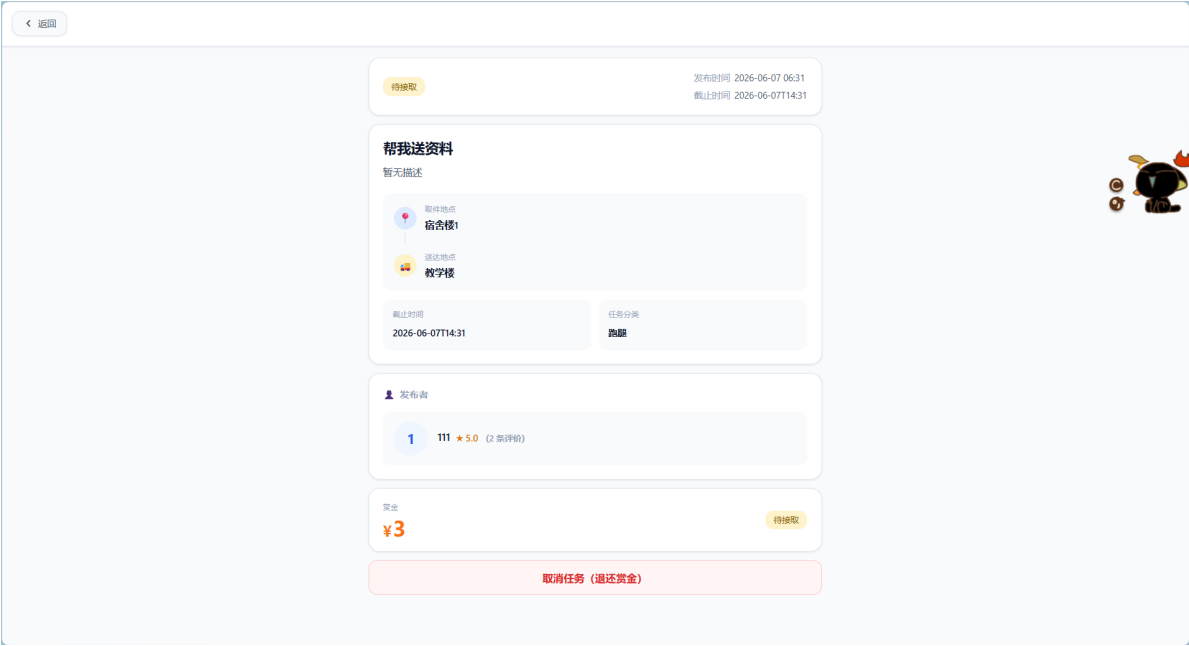
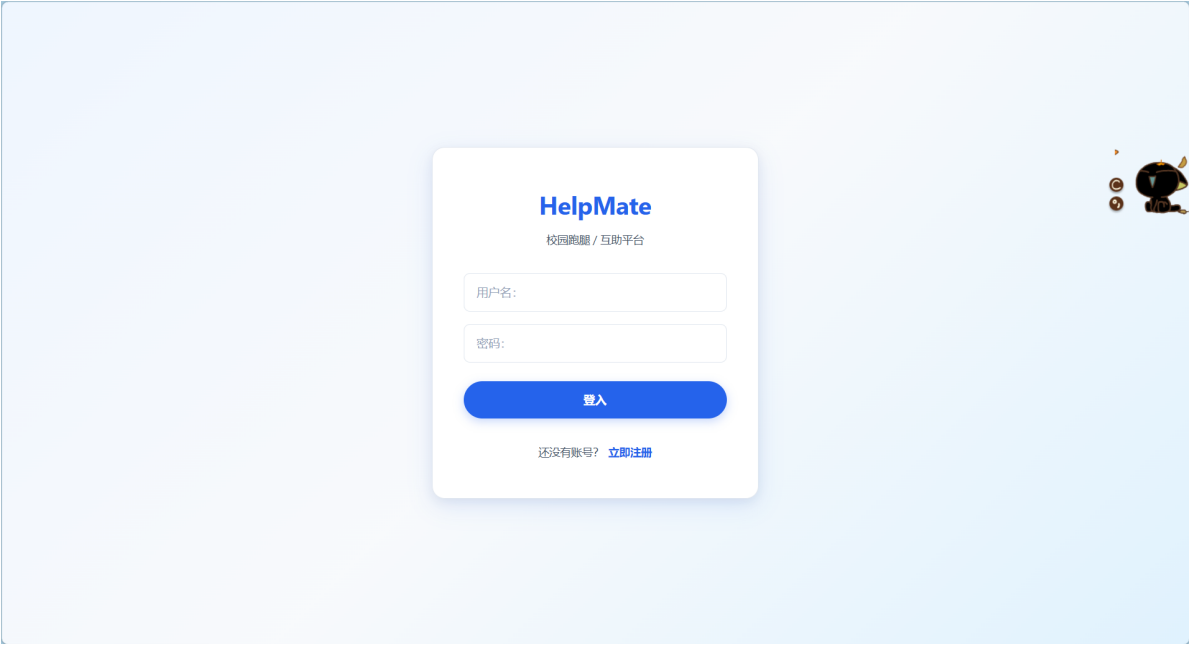
首页（Home.jsx）： 顶部导航栏含 AI 客服、订单、聊天、我的四个入口；搜索栏支持对标题/描述的本地关键词过滤；分类筛选按钮（全部/跑腿/代购/代拿/代办）切换时重新请求后端；任务以卡片网格展示，每张卡片含类别标签、赏金、标题、地点、发布者信息和发布时间；右下角悬浮"发布任务"按钮跳转发布页。

任务详情页（TaskDetail.jsx）： 展示完整任务信息和发布者评分；根据当前用户身份与任务状态动态渲染操作按钮（接单、完成、取消、联系、评价）；通过 `useParams` 读取路由中的 `taskId`，挂载时请求后端获取详情。

任务发布页（TaskPublish.jsx）： 见第二章设计说明，前端对应实现了类别卡片选择、双地点输入、快捷加价按钮和表单提交逻辑。

订单消息页（OrderMessage.jsx）： 汇总展示接单记录和发布任务的订单，分两区块展示，各自调用对应接口获取数据。

【截图：首页.png、任务详情页.png、登录页.png 等】

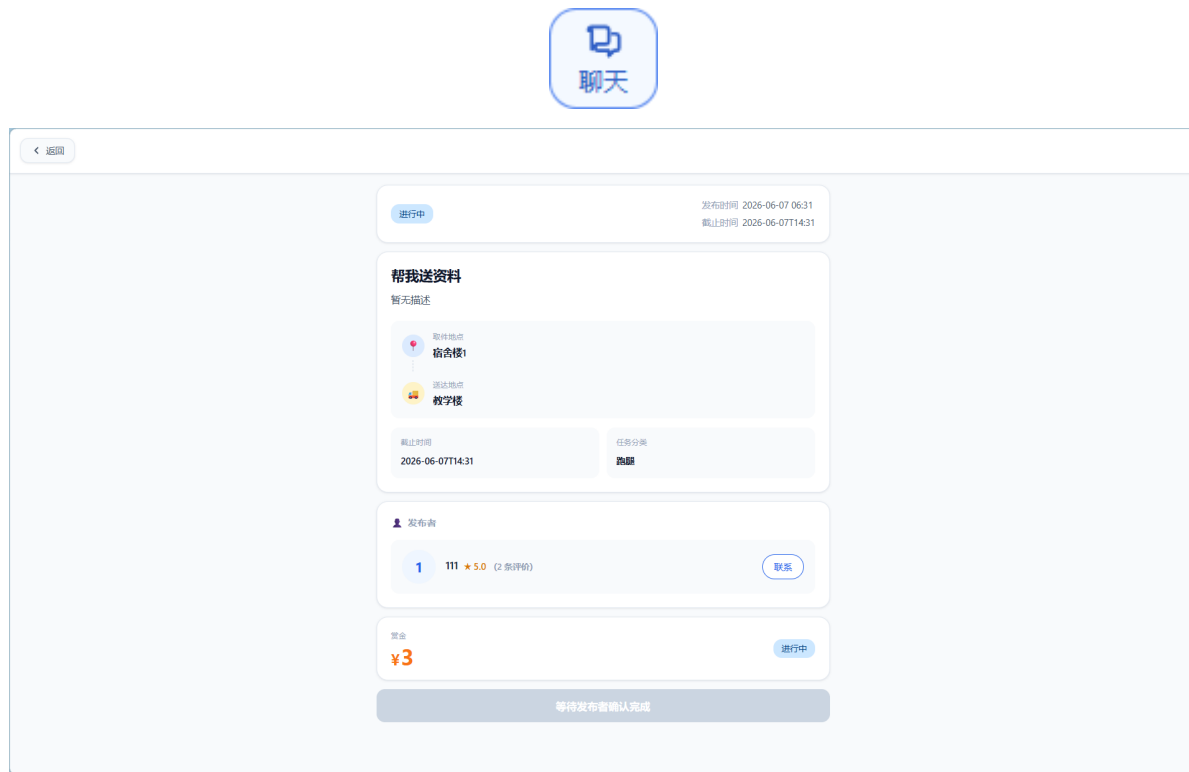


7.2 聊天功能（李丽丽）

功能入口

聊天有两个进入路径：首页导航栏"消息"入口，以及任务详情页在接取后出现的"联系发布者/接受者"按钮。按钮显示条件：任务状态为进行中或已完成，且当前用户是该任务的发布者或接取者。

【截图：首页聊天入口；任务详情页联系按钮】



会话列表（ChatList.jsx）

展示当前用户的所有聊天会话，每条显示对方头像、昵称、最新消息摘要、时间，以及未读数红点。进入页面时调用会话列表接口渲染，点击会话跳转聊天房间并同时调用标记已读接口，红点消失。



聊天房间（ChatRoom.jsx）

采用**3秒轮询** + **乐观更新**方案实现消息收发：

轮询：每隔3秒拉取最新消息列表，对比本地最后一条消息ID，将新增消息追加到列表末尾并自动滚动到底部。

乐观更新：用户点击发送后，先将消息以"发送中"状态立即插入本地列表，同时异步调用发送接口；成功后更新为已发送，失败则标红提示重试，用户感觉发送是即时的。

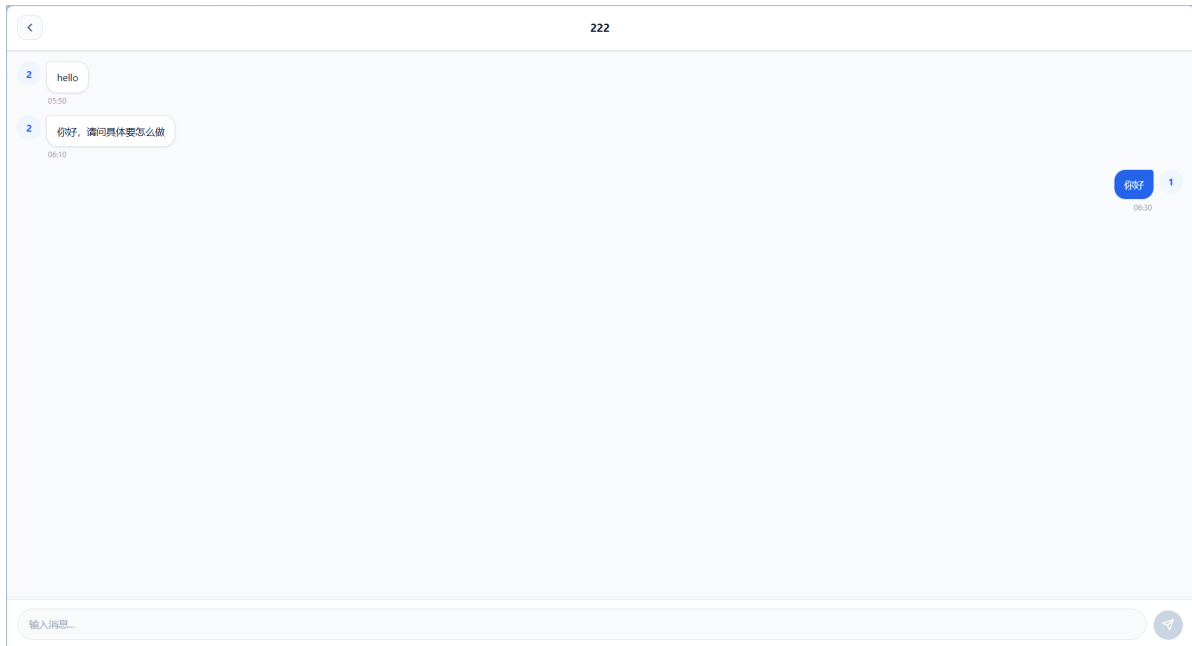
```
// 乐观更新核心逻辑（简化）
const handleSend = async () => {
  const tempMsg = { tempId: Date.now(), content: input, status: 'sending' };
  setMessages(prev => [...prev, tempMsg]);
  setInput('');
  try {
    const saved = await sendMessage(sessionId, input);
    setMessages(prev =>
```

```

    prev.map(m => m.tempId === tempMsg.tempId ? { ...saved, status: 'sent' } :
m)
    );
  } catch {
    setMessages(prev =>
      prev.map(m => m.tempId === tempMsg.tempId ? { ...m, status: 'failed' } :
m)
    );
  }
};

```

消息气泡布局：自己发的靠右、对方靠左；相邻消息间隔超过5分钟插入时间分割线；输入框固定在页面底部，消息区独立滚动。



7.3 评价功能重构（李丽丽）

重构前： 个人中心星级为硬编码满星，评语字段有数据但前端不展示，没有评价入口。

重构后：

- 评价入口移至任务详情页，任务完成后发布者和接取者均可看到"评价对方"按钮，提交后变为灰色不可重复
- 任务详情页展示发布者综合评分，接单前可了解对方信誉
- 封装动态 `StarRating` 组件，根据实际评分渲染实心星/半星/空心星，精确到0.5星

```

function StarRating({ score }) {
  const full = Math.floor(score);
  const half = (score - full) >= 0.3;
  const empty = 5 - full - (half ? 1 : 0);
  return (
    <span>
      {'★'.repeat(full)}{half ? '½' : ''}{ '☆'.repeat(empty)}
    </span>
  );
}

```

我收到的评价

★★★★★

2026-06-07 06:30

完成的很好

— 222

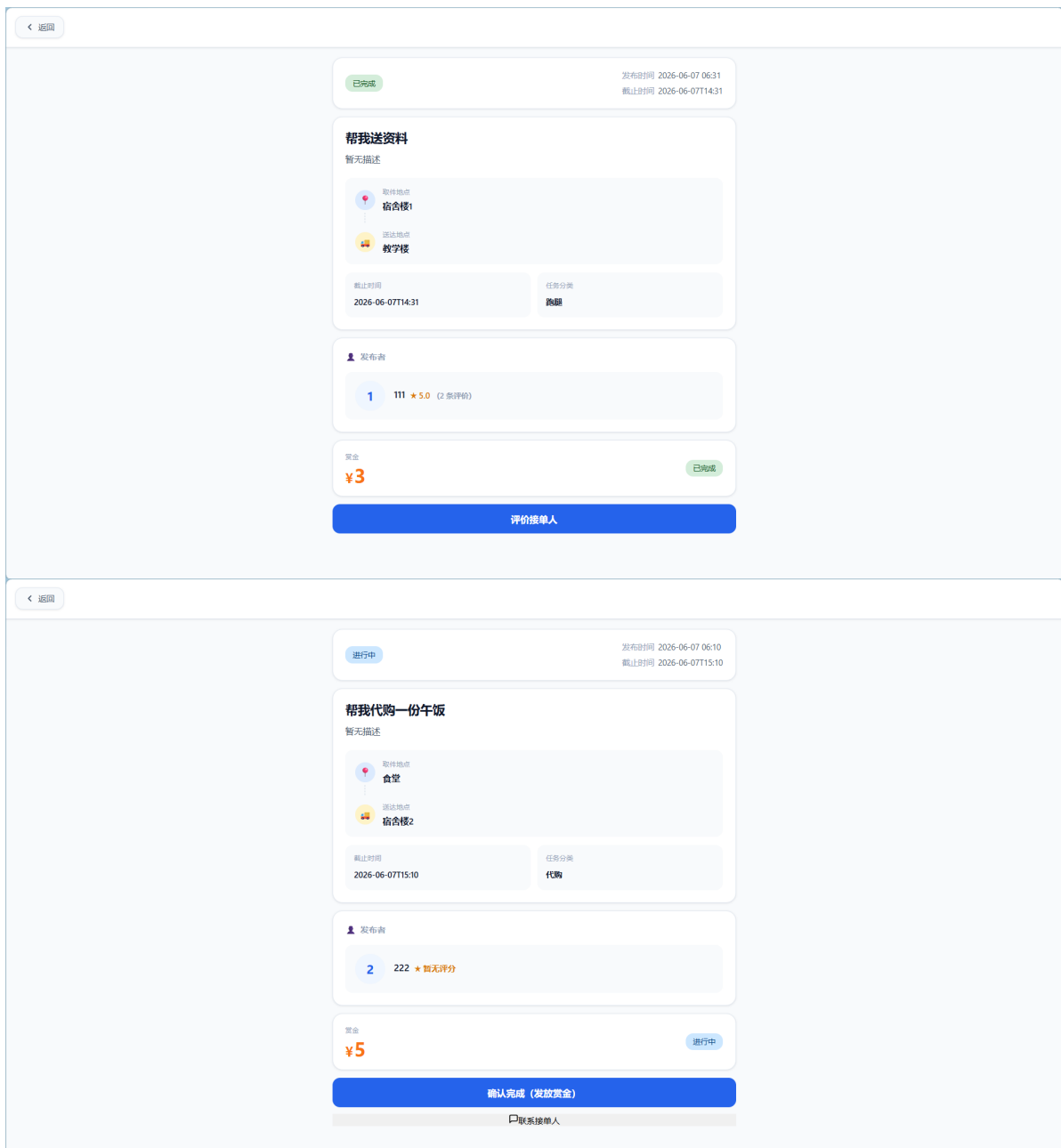
★★★★★

2026-05-19 11:16

不错，给钱很爽快

— 222

关闭



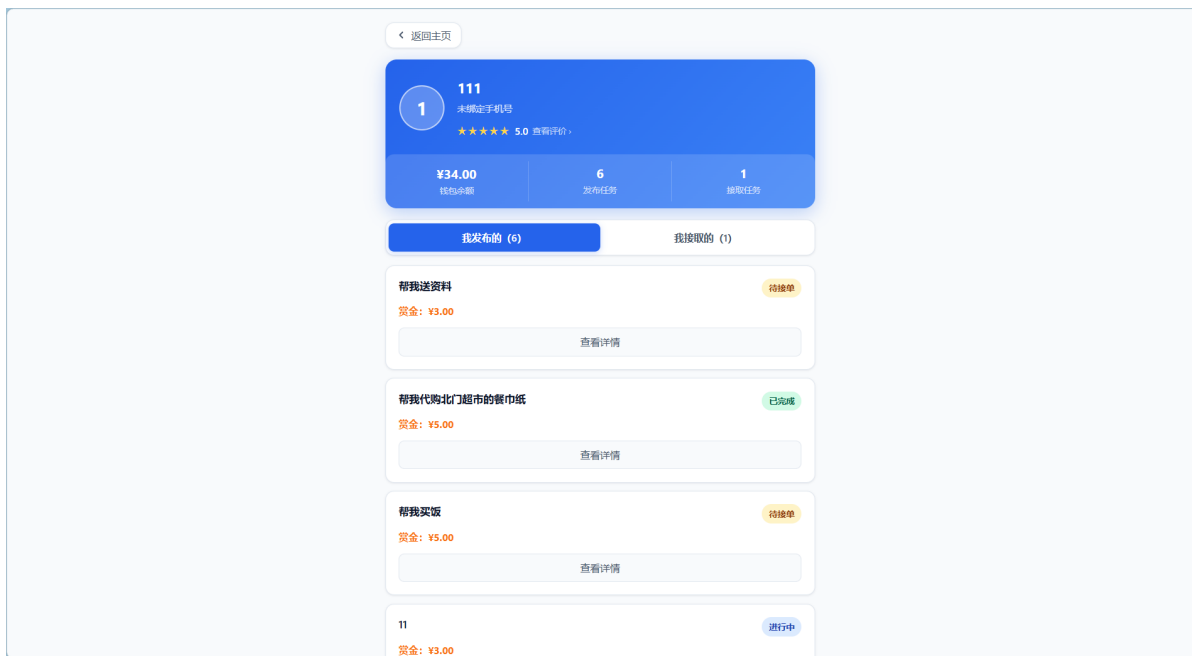
7.4 个人中心重构（李丽丽）

重构前：发布和接取任务混在一起、个人星级只能查看硬编码满星

重构后：

- 任务历史拆为"我发布的"和"我接取的"两个独立 Tab，各自支持按状态筛选
- 根据实际评分渲染实心星/半星/空心星，精确到0.5星，可查看具体文字评价

★★★★★ 5.0 查看评价 >



八、软件测试

本章分工：后端测试主责 → 商雨婷（李丽丽协助）；前端测试 → 陈晓彤。

8.1 后端测试（商雨婷）

后端测试基于 JUnit 5 + Mockito 框架，分为 Service 层单元测试和 Controller 层切片测试（`@WebMvcTest`）。

Service 层测试：使用 `@ExtendWith(MockitoExtension.class)` 加载 Mockito，通过 `@Mock` 注入所有依赖的 Mapper 和 Service，由 `@InjectMocks` 实例化被测 Service。测试覆盖的类包括：

- `UserServiceTest`：注册成功、用户名重复报错、密码被 BCrypt 加密（验证存库值以 `$2a$` 开头）、登录成功返回 Token、用户不存在报错、密码错误报错、不区分两种错误信息
- `TaskServiceTest`：发布任务返回 ID、初始状态校验、分页列表（有/无 category 筛选）、页参数透传、我的发布任务、按 ID 查任务（存在/不存在）、取消任务（成功/非发布者/非待接单状态）
- `OrderServiceTest`：接单成功（含通知推送）、接自己发布的任务报错、任务已被接单报错、完成订单（含赏金结算）、取消订单（含退款）、查我接的单、查我发布的单、查订单详情
- `walletServiceTest`：查余额、充值、余额不足提现报错、冻结赏金、释放赏金、退款、分页查流水
- `ReviewServiceTest`：提交评价（含评分更新）、重复提交报错、查用户评价列表、查用户资料
- `MessageServiceTest`：发送消息成功、给自己发消息报错、通知失败不影响主流程、查会话消息、查会话列表

Controller 层测试：使用 `@WebMvcTest` 切片加载，通过 `MockMvc` 发送 HTTP 请求，`@MockBean` 替换 Service 依赖，`@BeforeEach` 中绕过 `AuthInterceptor`（模拟返回 true 并注入 `userId=1`）。测试覆盖的 Controller 包括用户、任务、订单、评价、钱包、消息、通知和健康检查，共覆盖 40 余个请求路径，验证 HTTP 状态码、响应 JSON 字段和 Service 方法调用次数。

此外，`JwtUtilTest` 通过 `ReflectionTestUtils` 注入 secret 和 expiration，测试 Token 生成、解析和 `userId` 提取；`ResultTest` 测试所有静态工厂方法的 code/message/data 组合。

整体后端代码覆盖率（JaCoCo）达到 **48%+**，核心 Service 和 Controller 层覆盖率显著更高。

8.2 前端测试（陈晓彤）

前端测试基于 Vitest + React Testing Library，测试文件位于 `frontend/src/__tests__`。

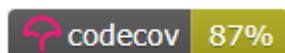
测试分为两类：**组件测试**使用 `render` 渲染 JSX，通过 `screen.getBy*` 断言 DOM 内容和交互行为；**工具函数/Mock API 测试**直接调用函数并断言返回值。所有涉及后端请求的测试均使用 `mockApi.js`（Mock 层），不发真实网络请求。

测试覆盖范围包括：

- `Login.test.jsx`：登录表单渲染、输入验证、提交成功跳转首页、错误提示展示
- `Register.test.jsx`：注册表单渲染与提交逻辑
- `Home.test.jsx`：任务列表渲染、分类筛选、搜索过滤、加载状态
- `TaskPublish.test.jsx`：发布表单验证与提交流程
- `mockApi.test.js`：Mock API 层的全部函数（登录、注册、任务增删改查、订单、评价、钱包、消息、通知等），共覆盖接口函数约20个
- `ErrorBoundary.test.jsx`：正常渲染子组件、错误时展示 fallback UI、重试/刷新按钮行为
- `logger.test.js`：日志级别过滤、控制台方法路由（info/warn/error）、结构化日志字段
- `metrics.test.js`：startTimer、recordApiCall、recordError、getSnapshot、resetStats、errorRate 计算
- `healthCheck.test.js`：前端健康检查、后端连通性探测（成功/失败）、定时检查启停

全部45个测试用例100%通过，前端代码覆盖率达到 **87%+**。

【截图：前端测试 `npm run test:coverage` 输出；Codecov 前端覆盖率报告】



8.3 后端测试协助（李丽丽）

李丽丽协助商雨婷编写了 `UserServiceTest`、`TaskServiceTest`、`UserControllerTest`、`TaskControllerTest` 中的部分测试用例，共协助编写约29个（与商雨婷测试用例有重叠）。后续独立编写了 `OrderServiceTest`、`ReviewServiceTest`、`WalletServiceTest`、`MessageServiceTest`、`OrderControllerTest`、`ReviewControllerTest`、`WalletControllerTest`、`MessageControllerTest`、`HealthAndNotificationControllerTest`、`JwtUtilTest`、`ResultTest`，共覆盖 70+ 个测试用例，将后端覆盖率从18%提升至48%以上。

九、CI/CD

本章分工： 后端 CI job → 商雨婷；前端 CI job → 陈晓彤；ci.yml 整合、冲突解决、Codecov Token 配置与 README 徽章 → 李丽丽。

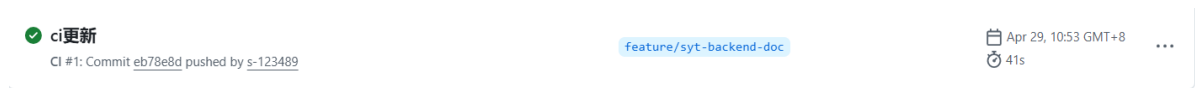
9.1 后端 CI job（商雨婷）

`ci.yml` 中后端 job 名称为 `Backend (Java / Maven)`，运行在 `ubuntu-latest`。构建步骤如下：

1. `actions/checkout@v4` 检出代码
2. `actions/setup-java@v4` 安装 JDK 17（Temurin 发行版），开启 Maven 依赖缓存（`cache: maven`），避免每次重新下载
3. 在 `backend/` 目录执行 `mvn -B test`，通过环境变量 `AI_API_KEY=dummy-key-for-ci` 注入占位凭据，使 AI 相关 Bean 能正常初始化（不发真实请求）

- 使用 `codecov/codecov-action@v4` 上传 JaCoCo 生成的覆盖率报告，flags 标记为 `backend`，`fail_ci_if_error: false` 保证覆盖率上传失败不阻断构建

后端 job 不配置 MySQL 服务容器，因为所有测试均为纯 Mock 测试，不连接真实数据库。

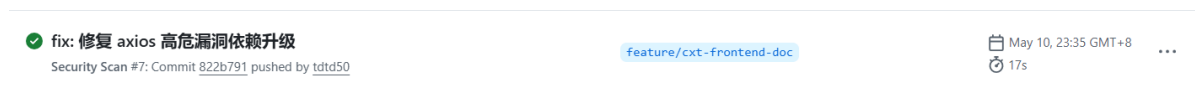


9.2 前端 CI job（陈晓彤）

`ci.yml` 中前端 job 名称为 `Frontend (React / Vite)`，运行在 `ubuntu-latest`。构建步骤如下：

- `actions/checkout@v4` 检出代码
- `actions/setup-node@v4` 安装 Node.js 20
- 在 `frontend/` 目录执行 `npm install` 安装依赖，并执行 `chmod -R +x node_modules/.bin` 确保 CLI 工具可执行
- 执行 `npm run lint`，ESLint 要求零警告零错误，若有 lint 问题直接构建失败
- 执行 `npm run test:coverage` 运行全部 Vitest 测试并生成 `coverage/lcov.info`
- 使用 `codecov/codecov-action@v4` 上传 `frontend/coverage/lcov.info`，flags 标记为 `frontend`
- 执行 `npm run build` 验证生产构建无报错

前端 job 与后端 job 并行执行，互不依赖，缩短整体 CI 时间。

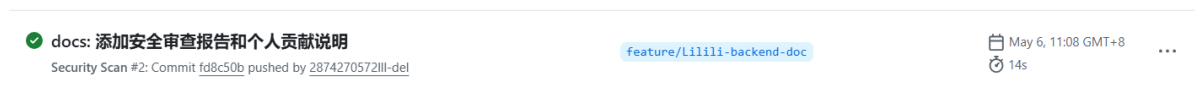


9.3 ci.yml 整合与冲突解决（李丽丽）

后端 CI job 和前端 CI job 由两位成员在不同分支分别开发，合并时产生了 Merge Conflict。李丽丽负责将两个 job 整合到同一个 `ci.yml` 文件中，解决冲突的关键是将两者的环境配置隔离到各自 job 内（后端需要 Java 17 + Maven 缓存，前端需要 Node.js + npm 缓存），避免相互污染，同时统一了 `on` 触发条件，确保 push 到 main/develop 分支或提交 PR 时两个 job 并行触发。

9.4 Codecov Token 配置与 README 徽章（李丽丽）

在 GitHub 仓库 Secrets 中配置了 `CODECOV_TOKEN`，并在 CI 工作流中添加覆盖率报告上传步骤，使前后端覆盖率数据可以上传到 Codecov。配置完成后在 `README.md` 中添加了覆盖率徽章，使项目测试状态在仓库首页直观可见。



HelpMate — 校园跑腿 / 互助平台

业务逻辑完整，贴近校园生活。技术栈：React / Vite + Spring Boot。
创新点：订单状态机 + 钱包支付的组合。



十、安全审查

本章分工：后端安全修复 → 商雨婷；前端安全修复 → 陈晓彤；CI Gitleaks 密钥扫描 (security.yml) → 李丽丽。

10.1 后端安全修复（商雨婷）

密码安全：所有用户密码在存库前使用 BCrypt 加密，strength 默认10，不可逆，即使数据库泄露也无法反推原始密码。

敏感配置环境变量化：JWT 密钥 (jwt.secret)、数据库连接密码、AI API Key 均通过环境变量注入，application.yml 中只写占位符，不含真实凭据，CI 中通过 GitHub Secrets 或明文 dummy 值注入。

安全响应头：webMvcConfig 中注册了 OncePerRequestFilter，对所有响应自动添加 X-Content-Type-Options: nosniff (防 MIME 嗅探)、X-Frame-Options: DENY (防点击劫持)、X-XSS-Protection: 1; mode=block (旧版浏览器 XSS 过滤)、Referrer-Policy: strict-origin-when-cross-origin。

账号枚举防护：登录接口无论用户名不存在还是密码错误，均返回相同错误信息"用户名或密码错误"，防止攻击者通过错误信息区分账号是否存在。

SQL 注入防护：所有数据库查询通过 MyBatis-Plus 的 LambdaQueryWrapper 或 Mapper 接口参数化执行，不拼接 SQL 字符串，框架层面杜绝 SQL 注入。

10.2 前端安全修复（陈晓彤）

XSS 防护：React 框架对所有通过 JSX 渲染的字符串自动转义 HTML 特殊字符，无 dangerouslySetInnerHTML 使用，不存在 XSS 注入点。

Token 存储：JWT Token 存储在 localStorage 中 (键名 helpmate_token)，而非 Cookie，避免 CSRF 攻击。前端代码中无任何敏感凭据 (API Key、数据库密码等) 硬编码，后端地址通过 Vite 环境变量 VITE_API_BASE_URL 注入。

依赖漏洞扫描：CI security.yml 中集成了 npm audit --audit-level=high，对前端 npm 依赖进行高危漏洞扫描，发现高危漏洞时 job 失败阻止合并。

构建产物处理：Vite 生产构建输出到 dist/ 目录，该目录已加入 .gitignore，不提交到仓库；Dockerfile 第二阶段只复制 dist/ 到 Nginx 静态目录，不包含源码或 node_modules。

10.3 Gitleaks 扫描：问题背景（李丽丽）

安全审查阶段李丽丽对仓库历史提交进行了审查，发现早期存在将 API Key 等敏感凭据硬编码提交到仓库的风险。即使后续通过环境变量修复了代码本身，历史 commit 中仍可能保留敏感信息，因此在 CI 层面引入了自动化密钥扫描。

10.4 Gitleaks 扫描配置（李丽丽）

在 .github/workflows/security.yml 中配置了两个 job：

Gitleaks 密钥扫描：使用官方 gitleaks/gitleaks-action@v2，fetch-depth: 0 拉取完整历史。Gitleaks 通过正则规则匹配常见敏感信息模式 (API Key、GitHub Token、JWT 密钥、数据库连接字符串明文密码等)，扫描全部提交历史。若扫描发现敏感信息，job 标记失败，PR 被阻止合并。

前端依赖漏洞扫描：额外增加了 npm audit --audit-level=high 的独立 job，专门对前端依赖进行高危漏洞扫描，与 Gitleaks 并行运行，保证代码和依赖双重安全检查。

security.yml 在每次 push 和 PR 时均触发，不区分分支，保证任何一次提交都经过安全扫描。

Security Scan

✓ Merge branch 'feature/Lilili-backend-doc' into develop #3

Re-run all jobs

...

Summary

All jobs

gitleaks

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

2874270572III-del pushed b509e02 develop

Status	Total duration	Artifacts
Success	13s	1

security.yml

on: push

gitleaks

9s

[- +]

gitleaks summary

...

No leaks detected ✓

[Job summary generated at run-time](#)

Annotations

1 warning

gitleaks

Node.js 20 actions are deprecated. The following actions are run...

Show more

十一、Docker 部署

本章分工： 后端 Dockerfile 多阶段构建 → 商雨婷；前端 Dockerfile + nginx.conf → 陈晓彤；compose.yml → 商雨婷·陈晓彤（李丽丽协助）；/health 端点与健康检查 → 商雨婷·陈晓彤。

11.1 后端 Dockerfile 多阶段构建（商雨婷）

后端 Dockerfile 分两个阶段，最终镜像只包含运行时组件：

构建阶段（builder）： 基础镜像为 `maven:3.9-eclipse-temurin-17-alpine`。先仅复制 `pom.xml` 并执行 `mvn dependency:go-offline` 下载依赖（利用 Docker 层缓存，只要 pom 不变就不重新下载），再复制源码执行 `mvn package -DskipTests` 生成 jar 包，跳过测试是因为 CI 阶段已单独运行测试。

运行阶段： 基础镜像切换为轻量的 `eclipse-temurin:17-jre-alpine`（只含 JRE，不含 JDK 和 Maven），只复制 builder 阶段产出的 jar 文件，镜像体积大幅减小。创建非 root 的 `appuser` 运行应用，遵循最小权限原则。

健康检查配置为每30秒通过 `wget` 请求 `http://localhost:8080/health`，失败3次认为服务不健康。Compose 中其他服务的 `depends_on` 通过 `condition: service_healthy` 等待后端就绪后再启动。

11.2 前端 Dockerfile + nginx.conf (陈晓彤)

前端 Dockerfile 同样分两阶段：

构建阶段 (builder)：基础镜像为 `node:20-alpine`。先复制 `package.json` 和 `package-lock.json` 执行 `npm ci` (比 `npm install` 更严格，严格按 lock 文件安装)，再复制全部源码执行 `npm run build`，产出 `dist/` 静态文件目录。

运行阶段：基础镜像切换为 `nginx:1.27-alpine`，复制自定义 `nginx.conf` 和构建产出的 `dist/`，以非 root 的 `appuser` 运行 Nginx 进程。

`nginx.conf` 的关键配置包括：监听8080端口（而非默认80，避免在容器内使用特权端口）、`root /usr/share/nginx/html` 服务静态文件、`try_files $uri $uri/ /index.html` 处理 SPA 的客户端路由 fallback（任何不匹配静态文件的路径都返回 `index.html`，由前端路由接管）、`/api/` 路径反向代理到后端容器 `http://backend:8080`。

健康检查同样通过 `wget` 请求前端的 `/health` 路径（Nginx 返回200即可）。

11.3 compose.yaml / compose.prod.yaml (李丽丽协助)

李丽丽协助完善了 `compose.yaml`（开发环境）和 `compose.prod.yaml`（生产环境）中的配置。两个文件的主体由商雨婷编写，陈晓彤新增了 frontend 服务配置，李丽丽参与了配置调试与验证。

`compose.yaml` 定义三个服务：`db` (MySQL 8.0)、`backend` (Spring Boot)、`frontend` (Nginx + React)。服务依赖链为 `frontend → backend → db`，全部通过 `depends_on.condition: service_healthy` 等待前置服务就绪。后端所需的 DB 连接串、JWT 密钥、AI 配置均通过 `environment` 注入，其中 `AI_API_KEY` 支持从宿主机环境变量透传（`${AI_API_KEY:-}`）。数据库数据通过 Named Volume `dbdata` 持久化。

11.4 /health 端点与健康检查配置 (商雨婷、陈晓彤)

后端 /health 端点：`HealthController` 提供 `GET /health` 接口，该路径在 `AuthInterceptor` 的排除列表中，无需认证即可访问，返回 `{"status":"ok"}` 和 HTTP 200。后端 Dockerfile 的 `HEALTHCHECK` 指令、`compose.yaml` 的后端 `healthcheck` 均通过 `wget` 请求此端点判断后端是否就绪。

前端健康检查：前端 Nginx 容器的健康检查也请求 `http://127.0.0.1:8080/health`，Nginx 将此路径作为静态文件请求处理，返回200即视为正常（`try_files` 会返回 `index.html`，HTTP 状态为 200）。

三个服务均配置 `interval: 30s`、`timeout: 10s`、`retries: 3`，确保启动顺序可靠，避免后端未就绪时前端已开始转发 API 请求。

十二、总结

本章分工： 全体成员共同撰写，未完成与待改进部分各人据实填写。

12.1 已完成功能

李丽丽部分：

- 任务发布页 UI 主设计
- 数据库全部8张表的设计（含 Flyway 分版本迁移策略）
- 用户认证与任务管理 API 规范设计；Apifox 测试5个用例
- **聊天功能（前后端全部）**：会话列表、聊天房间（3秒轮询+乐观更新）、消息相关后端接口
- **评价功能重构**：动态星级、评语展示、任务详情页评价入口与发布者信誉展示
- **个人中心重构**：发布/接取任务分区、可查看评价
- MyMetaObjectHandler 时间戳自动填充
- CORS OPTIONS 预检请求修复
- CI 整合（ci.yml合并冲突解决）、Codecov 配置、README 徽章
- Gitleaks CI 密钥扫描（security.yml）
- 后端测试协助（70+ 测试用例，覆盖率从18%提升至48%+）

商雨婷部分：

- 登录页、配色方案 UI 设计
- 用户模块（注册/登录，BCrypt 加密，JWT 签发）
- 任务模块（发布/列表/详情/接单/完成/取消，状态机，并发控制）
- DeepSeek AI 智能客服功能集成
- 后端测试主责（UserServiceTest、TaskServiceTest 等）
- 后端 Dockerfile 多阶段构建
- 后端安全修复（安全响应头、密码加密、环境变量化）
- compose.yaml 主体编写

陈晓彤部分：

- 任务详情页/个人中心页/订单页 UI 设计
- 前端全部基础页面实现（登录/注册/首页/任务详情/任务发布/订单消息）
- 前端测试（45例，100%通过，覆盖率87%+）
- 前端 CI job（lint + test:coverage + build）
- 前端 Dockerfile + nginx.conf（多阶段构建+SPA路由fallback+反向代理）
- 前端安全修复（XSS防护、Token存储、依赖扫描）

12.2 未完成与待改进部分

李丽丽部分：

- **聊天实时性**：当前3秒轮询在高并发时存在服务器压力，更优方案是 WebSocket 实时推送，因时间限制未完成，保留为后续优化方向
- **钱包安全性**：充值未接入真实第三方支付，属演示用途，生产环境需引入支付验签机制

商雨婷部分：

- DeepSeek AI 功能在 CI 中使用 dummy key，未覆盖真实 AI 调用的集成测试
- 后端测试覆盖率距生产级目标（核心业务 80%+）仍有差距，部分边界场景未覆盖

陈晓彤部分：

- 前端暂无端到端（E2E）测试，仅有单元/组件测试，联调后的完整业务流程未自动化验证
- 部分页面在移动端小屏（< 375px）下存在布局溢出，未做充分的响应式适配

本文档基于实际项目代码整理，各章节描述与代码实现保持一致。技术内容经 AI 辅助组织语言后经成员核对修改，所有描述均与仓库代码对应。

